1. What are the advantages of Polymorphism?

- **Code Reusability**: Polymorphism allows a single interface or superclass type to represent multiple forms, enabling the reuse of code across different classes. For example, a method accepting a superclass type can work with any subclass object.
- **Flexibility and Extensibility**: New subclasses can be added without modifying existing code, as long as they conform to the superclass or interface. This makes systems easier to extend.
- **Simplified Code Maintenance**: Polymorphism reduces the need for duplicate code by allowing a single method to handle different types, making the codebase easier to maintain.
- **Improved Readability**: By using a common interface or superclass, code becomes more intuitive and organized, as the same method call can work with different types.

2. How is Inheritance useful to achieve Polymorphism in Java?

- **Superclass Reference for Subclass Objects**: Inheritance allows a superclass reference to point to a subclass object. For example, if Media is a superclass and DigitalVideoDisc is a subclass, you can write Media media = new DigitalVideoDisc(...). This enables polymorphic behavior because the actual method called (e.g., media.toString()) depends on the runtime type (DigitalVideoDisc).
- **Method Overriding**: Inheritance enables subclasses to override methods from the superclass. At runtime, Java uses dynamic method dispatch to call the overridden method of the actual object type, achieving runtime polymorphism (e.g., DigitalVideoDisc overriding toString() from Media).
- **Common Interface**: Inheritance provides a common type (the superclass) that can be used to interact with different subclasses uniformly, such as in collections (List<Media>) or method parameters (void play(Media media)).

3. What are the differences between Polymorphism and Inheritance in Java?

- **Definition**:
  + **Polymorphism**: The ability of a single method or object to take on multiple forms, either through method overriding (runtime polymorphism) or method overloading (compile-time polymorphism).
  + **Inheritance**: A mechanism where a class (subclass) inherits fields and methods from another class (superclass), establishing an "is-a" relationship.
- **Purpose:**
  + **Polymorphism**: Focuses on enabling flexibility by allowing different types to be treated uniformly through a common interface or superclass.
  + **Inheritance**: Focuses on code reuse and establishing a hierarchical relationship between classes, where subclasses inherit behavior and state from a superclass.
- **Mechanism**:
  + **Polymorphism**: Achieved through method overriding (runtime) or overloading (compile-time), often relying on inheritance or interfaces.

+ **Inheritance**: Achieved by using the extends keyword to create a subclass that inherits from a superclass.

- **Scope**:
  + **Polymorphism**: Can exist without inheritance (e.g., through interfaces or method overloading).
  + **Inheritance**: Is a prerequisite for runtime polymorphism via method overriding but is not required for compile-time polymorphism.
- **Example**:
  + **Polymorphism**: A method play(Media media) can call media.play() on a DigitalVideoDisc or CompactDisc object, with the correct play() method executed based on the runtime type.
  + **Inheritance**: DigitalVideoDisc inherits getTitle() from Media, reusing the method without redefining it.